



Kata Senior - Fullstack/Cloud | Renombramiento Inteligente de Archivos en S3

🏆 Reto Senior – Renombramiento Inteligente de Archivos en S3

🧠 Kata Senior Fullstack Cloud

Renombramiento Inteligente de Archivos en S3

📌 Contexto

Una entidad financiera recibe diariamente archivos en un bucket de Amazon S3 desde múltiples sistemas origen.

Estos archivos llegan con nombres no estandarizados, que incluyen:

- Prefijos variables por sistema origen
- Tipos de producto financiero
- Fechas embebidas en el nombre del archivo
- Convenciones inconsistentes entre entidades

El banco requiere una solución que permita **normalizar automáticamente los nombres de los archivos** según un estándar definido.

🎯 Objetivo

Construir una solución Fullstack Cloud que:

- Lea una lista de archivos desde un bucket S3 (simulado o real).
- Identifique patrones de nombre de archivo de origen.
- Aplique reglas de transformación.
- Genere el nombre estandarizado esperado por el banco.

- Muestre el resultado del proceso y posibles errores de mapeo.
-

Entrada (Origen - Entidad externa)

Los archivos llegan con nombres como:

- PHO_CD_DES_AAAAMMDD
- PHO_SV_AAAAMMDD
- PHO_ML_UTIL_AAAAMMDD.txt
- cuotas_bdb_AAAAMMDD.txt
- activos_inmob_bdb_AAAAMMDD.txt
- PrendasPajaro.txt

Estos nombres pueden variar en:

- Prefijos
 - Sufijos
 - Extensión del archivo
 - Formato de fecha embebido (AAAAMMDD , AAAADDMM , etc.)
-

Salida esperada (Destino - Banco)

El sistema debe transformar los nombres a un estándar como:

- 01_Estructura CDT Desmaterializado_20260430
 - 03_Estructura Cuenta Ahorros_20260430
 - 13_CUOTAS Activos
 - 14_Hipotecaria
 - 37_Leasing_Vehículo
-

Reglas de transformación

1. Normalización de fecha

- Detectar patrones como:
 - AAAAMMDD
 - AAAADDMM
 - Reemplazarlos por una fecha estándar (ej: 20260430)
-

2. Mapeo de estructuras

El sistema debe soportar un **catálogo de mapeo configurable**:

Archivo Origen	Layout Destino
PHO_CD_DES_*	01_Estructura CDT Desmaterializado
PHO_SV_*	03_Estructura Cuenta Ahorros
PHO_CK_*	04_Estructura Cuenta Corriente
PHO_ML_*	13_CREDITOS / CUOTAS / ACTIVO según subtipo
garantias_*	14_* (Solo Firma, Codeudor, etc.)
activos_*	37_Leasing_*

3. Manejo de variaciones

El sistema debe poder:

- Identificar archivos sin extensión o con `.txt`
- Manejar archivos sin fecha embebida
- Resolver múltiples patrones similares
- Marcar como **"No mapeado"** cuando no exista regla

Funcionalidades

1. Lectura de archivos desde S3

- Listar archivos disponibles en el bucket (simulado o real).

2. Motor de transformación

- Aplicar reglas de renombramiento.
- Separar lógica de reglas vs procesamiento.

3. Resultado del proceso

Mostrar:

- Archivo origen

- Nombre transformado
 - Estado (Transformado / Error / No mapeado)
 - Regla aplicada
-

4. Panel de control

- Total de archivos procesados
 - Archivos transformados correctamente
 - Archivos con error
 - Archivos no mapeados
-

Componente Cloud

La solución debe poder ejecutarse en AWS (Free Tier), usando opcionalmente:

- S3 (entrada de archivos)
 - Lambda o Backend API
 - DynamoDB o PostgreSQL
 - SQS (opcional para procesamiento asíncrono)
-

Nivel Senior (lo que se evalúa)

- Diseño de un motor de reglas flexible
 - Separación entre configuración y lógica
 - Manejo de patrones de texto complejos
 - Capacidad de escalar reglas sin reescribir código
 - Diseño cloud-native
 - Manejo de casos no estructurados
-

Interfaz y Experiencia

- Interfaz para administrar reglas de mapeo
 - Versionamiento de reglas
 - Reprocesamiento de archivos
 - Simulación de carga en lote desde S3
-

Entregables

- Código fuente
 - README de ejecución
 - Docker Compose
 - Diagrama de arquitectura
 - Evidencia de ejecución
 - (Opcional) despliegue en AWS Free Tier
-

Alcance esperado

El reto debe poder resolverse en **4 a 6 horas efectivas**, priorizando diseño, arquitectura y claridad del motor de reglas sobre la cantidad de funcionalidades.

Tiempo de exposición 8 min

Lo que queremos observar

Más que la cantidad de funcionalidades, buscamos entender cómo el candidato:

- Diseña una solución mantenible y escalable.
 - Maneja procesos y validaciones complejas.
 - Implementa procesamiento asíncrono.
 - Aplica buenas prácticas de desarrollo Fullstack y Cloud.
 - Justifica sus decisiones técnicas y arquitectónicas.
-

Consideraciones de Seguridad

- No subir credenciales reales ni secretos a repositorios públicos.
 - Usar datos simulados en todos los ejemplos.
 - Versionar el código en repositorios personales.
 - Se recomienda el uso de `.gitignore` para excluir archivos sensibles.
-

 ¡Atención, equipo! Importante mensaje sobre seguridad en repositorios 

¡Importante!

 ¡Atención, equipo!  . General

Queremos recordarles algo **MUY IMPORTANTE** para mantener la seguridad de nuestra información y la del banco. Recientemente, tuvimos un incidente porque en una **Kata de desarrollo** se usó un repositorio real de la organización y accidentalmente se expuso un **Secret Key** en GitHub. 🤖

Esto representa un **riesgo grave** para la seguridad, por lo que necesitamos su ayuda para evitarlo en el futuro.

📌 Recomendaciones clave:

- ♦️ **No usen repositorios reales del banco en Katas.**
 - Las Katas son para práctica, ¡usen repositorios personales, no usar sandbox del banco pueden usar sandbox personales!
- ♦️ **Nunca suban credenciales, keys o información sensible.**
 - Si necesitan datos de prueba, usen valores ficticios (ej: `"token_ejemplo_123"`).
- ♦️ **✓ Verifiquen antes de hacer commit/push.**
 - Usen herramientas como `git-secrets` o `.gitignore` para evitar subir archivos críticos.
- ♦️ **🛡 Si ven algo raro, repórtenlo de inmediato.**

💡 Alternativas seguras para Katas:

- **GitHub Personal:** Crear un repo en sus cuentas personales (no vinculado al banco).
- **Entornos de prueba:** Usar bases de datos o APIs mock (como Mockaroo, JSON-Server).
- **Datos falsos:** Siempre que sea posible, generen datos simulados.

☀️ ¡Juntos protegemos nuestra seguridad! ☀️

Este tipo de errores pueden tener consecuencias serias, pero con buenas prácticas los evitamos. ¡Gracias por su colaboración!

¡Éxitos con el Reto! 🚀